

AIEN Research Plan

Building a sovereign synthesis intelligence from a small, inspectable lineage that learns through verified programs and machine evidence.

Drake Stapleton

drake@aenos.com

<https://www.aenos.com>

THE THESIS

Intelligence should be allowed to propose. Evidence should decide.

AIEN asks whether a machine intelligence can be built from a small, inspectable software lineage and learn by creating, testing, measuring, and reusing verified procedures, rather than depending permanently on an opaque tower of inherited software or treating a model's confidence as truth.

M0-M18

Implemented foundation

Canonical doctrine, minimal bootstrap, formal semantics, direct AArch64 realization, self-hosting, verification, program synthesis, procedural library learning, machine graphs, and Blackwell accelerator work have qualifying evidence at their stated scopes.

M19

Reopened honestly

The resident accelerator implementation landed, then independent review found evidence-integrity and runtime-correctness gaps. M19 is now in corrective requalification and is not complete.

M20-M40

Defined research path

The remaining roadmap covers sovereign tensor training, AIEN_0, guided synthesis, sealed scientific discovery, resident cognition, continual learning, autonomy, and succession.

The central bet

Durable intelligence should live in inspectable procedures, verified relationships, measured machine facts, and a governed memory of evidence. Neural weights may help search, but they do not receive final authority to declare their own answers correct.

What this is

A research program for a sovereign machine that can translate intent into formal meaning, discover a physical realization, verify it, act, learn from the result, and improve its methods.

What this is not

It is not merely an operating system, and AIEN V2 is not a conventional large language model trained by next-token prediction over internet-scale human prose.

This PDF is the dated 27 September 2026 research brief, not the live authority. Current status and sequencing remain governed by the canonical public roadmap and execution plan. Qualification scope matters: QEMU, hardware-boundary, and physical-silicon results are not interchangeable.^[1-4]

THREE DIFFERENT STATEMENTS

What exists, what is being repaired, and what remains a target

Class	What can be said now	Boundary
ACCOMPLISHED	M0 through M14 are recorded complete. M15 is hardware-boundary qualified. M16 is correctively requalified. M17 and M18 are qualified on DGX Spark GB10 silicon.	Each statement is limited to the gate and environment named in the roadmap.
ACCOMPLISHED	atlas.bin is recorded as 1,480 bytes. Its audit reconciles 301 instructions and 69 read-only-data words, and its QEMU harness refused 268 mutation cases.	Native Atlas hardware qualification remains pending.
ACCOMPLISHED	Omega has concrete code and receipts for semantic identity, direct AArch64 generation, self-hosting, verification, deterministic synthesis, reusable program libraries, abstraction discovery, adaptive realization, machine modeling, and accelerator realization.	These are bounded components, not proof of general intelligence.
ACTIVE	M19 corrective qualification: immutable receipts, probed hardware identity, full allocation lifecycle, exactly-once completion behavior, race testing, and a long silicon soak.	The prior implementation and 175-gate run do not close the reopened milestone.
ACTIVE	Forge boundary definition, AIENOS trust and identity work, Omega floating-point numerical closure, and composition interfaces.	These lanes must converge before AIEN_0 is promoted.
TARGET	Sovereign training, a learned search guide, guided synthesis, sealed discovery worlds, resident cognition, scientific autonomy, and succession.	No sovereign training has occurred. AIEN_0 and M20 through M40 remain planned.

The discipline is simple: never turn a target into a demonstration by changing the tense.

Evidence outranks narrative

A passing receipt can support only the exact claim it tested. A QEMU result is not a silicon result. A merged implementation is not a closed milestone when corrective review has reopened it.

Correction is progress

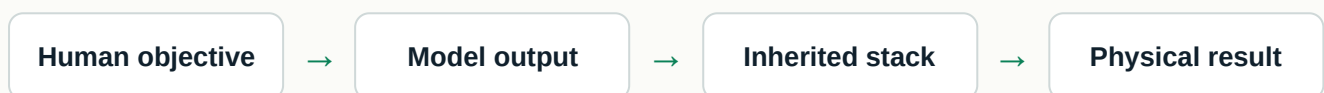
M19's reopening is not hidden as an embarrassment. It is evidence that the process can detect its own overstatement and force the work back through the gate.

WHY REBUILD FROM THE MACHINE UPWARD?

An intelligent agent can inherit corruption it cannot see

A modern agent appears to be a model responding to a prompt. In reality, its behavior rests on a vast chain of firmware, boot code, operating-system services, drivers, compilers, kernels, libraries, schedulers, protocols, and vendor runtimes. Any layer can distort a result.

The failure does not have to look dramatic. A small memory error, race, stale handle, numeric difference, driver edge case, or bad tool result can appear as ordinary model unpredictability. If the surrounding stack is not measured, the agent may learn from a corrupted observation and carry that error forward.



Without explicit checks, uncertainty accumulates across every hidden layer.

The conventional response

Trust mature components, patch failures, add monitoring, and accept that the whole stack is too large to inspect as one lineage.

The AIEN response

Shrink the permanent trusted lineage, separate meaning from representation, verify boundaries, record machine evidence, and make reconstruction part of the design.

Research question

Can an intelligence be built from a small, inspectable lineage that learns through verified program synthesis and machine evidence rather than relying primarily on capabilities encoded in learned parameters derived from large external corpora?

Scope of sovereignty

Sovereignty does not mean pretending no outside tool was ever used. It means the installed lineage can eventually reconstruct and operate its trusted path without depending on an incumbent operating system, opaque runtime, or external vendor service.

Meaning of trust

Trust is not a feeling and not a brand name. It is the smallest set of assumptions that must be accepted before a result can be reproduced, audited, or rejected.

THE TESTABLE BET

Separate meaning, realization, authority, and evidence

H1. Meaning can outlive a machine

A computation can be represented as a formal statement of what must be true, independent of the instructions, memory layout, or accelerator schedule used today.

H2. Procedures can be durable knowledge

Successful programs can be verified, measured, versioned, reused, compressed into abstractions, and retired when better evidence appears.

H3. Learning can guide without ruling

A neural guide can rank promising search paths while formal checks and physical evidence retain authority over correctness.

H4. Hardware can answer back

The machine can expose real constraints and measurements so the system learns what actually works, not what a library or model merely predicts.

Research questions

RQ1	Can formal semantic identity survive across different machine realizations while preserving bounded numerical behavior?
RQ2	Can deterministic synthesis and library learning create reusable capability before any sovereign neural guide exists?
RQ3	Can AIEN_0 reduce time or search cost to a valid realization without changing the verifier's acceptance standard?
RQ4	Can a small trusted lineage discover and sustain competitive machine realizations on heterogeneous hardware?
RQ5	Can a multi-agent development process detect more defects and compound more useful process knowledge than a single-agent workflow?
RQ6	Can the system eventually propose a successor while independent checks, frozen tests, isolation, and rollback prevent self-approval?

Each question has a failure path. If a proposed capability cannot meet a frozen acceptance contract, it remains outside the trusted lineage.

A CHAIN OF EXPLICIT RESPONSIBILITIES

No layer is allowed to grade its own work

TARGET RESPONSIBILITY MODEL

AIEN Proposes goals, hypotheses, plans, and candidate changes.	OMEGA Defines precise meaning, constraints, and programs.	FORGE Realizes meaning for the available hardware.	AEGIS Checks invariants, bounds, contracts, and resulting state.	HARDWARE Performs physical state transitions.	EVIDENCE Records what happened and teaches the next cycle.
--	---	--	--	---	--

AIENOS	Trusted operating substrate beneath the intelligence and realization loop: identity, continuity, recovery, networking, and native inference.
---------------	--

ATLAS AWAKENS AIENOS FROM A MEASURED ROOT

ATLAS AWAKENS. AIEN PROPOSES. OMEGA DEFINES. FORGE REALIZES.
AEGIS VERIFIES. HARDWARE ACTS. EVIDENCE TEACHES.

The permanent separation

Omega says what must be true. Forge decides how a particular machine might make it true. Aegis checks the contract. This prevents a fast implementation from redefining the meaning it was supposed to preserve.

Bidirectional reality

Forge does not merely receive instructions. It reports physical limits, layouts, queue behavior, memory constraints, and measured alternatives back to Omega.

The trusted operating substrate

AIENOS owns the operating foundation beneath this loop. The research plan requires boot, identity, encrypted continuity, recovery, networking, and native inference to become demonstrated machine properties rather than diagrams.

Terminology note

Current doctrine calls the machine-realization layer **Forge**. Historical milestone identifiers and evidence still use **Physics**. Preserving those names in old receipts protects provenance rather than creating a second current architecture.

The public architecture authority defines the responsibilities and the public plan authority records the Forge naming rule.^[1,3]

A DIFFERENT LEARNING SUBSTRATE

Programs explain. Models suggest. Verification decides.

Omega is designed to preserve the meaning of a task separately from any one way of executing it. It can construct candidate programs, reject invalid candidates, realize valid ones for a machine, measure them, and keep successful procedures in a versioned library.

Typical foundation-model learning path, simplified

1. Collect a very large body of human-written text.
2. Train a model to predict likely next tokens.
3. Adjust behavior using human preference signals.
4. Use the resulting weights as the main store of capability.

AIEN V2 research path

1. Define a task and its invariants in Omega.
2. Generate candidate programs and search traces.
3. Verify semantics and execute candidates.
4. Record success, failure, cost, machine context, and evidence.
5. Promote useful procedures into a governed library.

What AIEN_0 is planned to do

AIEN_0 is planned as a deliberately small neural search guide trained on Omega-generated traces. Its job is to point synthesis toward promising branches. It does not grant capabilities, certify correctness, or override the verifier. If it cannot reduce search cost while preserving verification parity, it does not earn integration.

Knowledge has two forms

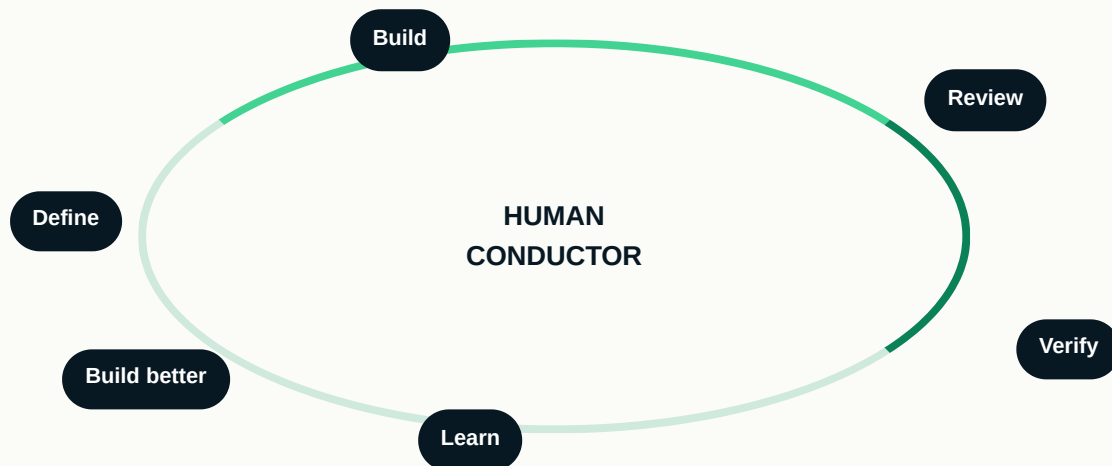
Procedural knowledge lives in verified programs and reusable abstractions. **Search intuition** may live in learned weights that help choose where to look next. Keeping these roles separate makes it possible to inspect what the system knows and test whether the guide actually helps.

Sovereign training has not occurred. M20 through M26, including AIEN_0, are planned milestones.^[2,4]

THE BUILD METHOD IS ALREADY THE PROTOTYPE

The Infinite Game turns disagreement into evidence

I lead this program by defining the objective, asking the hard questions, coordinating specialized intelligence, demanding proof, and deciding which claims are allowed to survive. Gemini, Claude, Codex, Grok, DeepSeek, and Muse contribute under that human direction.



Hierarchical orchestration

The conductor sets intent and the admission standard. Lead agents decompose work into bounded phases. Specialist agents implement, test, analyze, or document within a declared scope. Evidence returns upward before anything is accepted.

No single AI holds the complete picture

Context, authority, and duties are split across systems. Independent AI watchers review every commit and comment, challenge claims, and feed defects or contradictions back into the next cycle.

Diversity is a defense, not proof

Different providers can catch different mistakes, but they may still share training material, assumptions, or infrastructure. Cross-provider agreement is useful evidence. It is never treated as infallibility.

The loop never terminates

This is not a game played to win and finish. It is played to keep playing: build, verify, learn, and build better. Every finding updates later work, including succession beyond the original conductor.

“I’ve literally created what I am creating. I’m using its process and now I’m just trying to get myself out of it.”

The method mirrors the machine. Proposals are cheap. Verification is separate. Evidence becomes memory. Memory changes the next search. The current human coordination role is therefore a bootstrap function to be understood, formalized, measured, and eventually internalized by the verified system.

ONE CYCLE, PHASE ONE

From an objective to a reviewable candidate

01 State the objective

The conductor describes the result in ordinary language, including what must not happen. Ambiguity is surfaced before work begins.

02 Freeze the acceptance contract

Define success, failure, allowed inputs, environment, measurements, and evidence before seeing the answer. This prevents moving the goal after a result appears.

03 Admit the work

A Gatekeeper checks scope, authority, dependencies, safety boundaries, and path ownership. Work that cannot be bounded does not enter the execution queue.

04 Decompose hierarchically

Tier 1 leads divide the objective into phases. Tier 2 specialists receive small tasks, explicit interfaces, and no more context or authority than their task requires.

05 Assign multiple minds

Gemini, Claude, Codex, Grok, DeepSeek, and Muse contribute where their strengths fit. Competing designs are encouraged when the decision is consequential.

06 Build in isolated branches

Each candidate has a declared scope, change record, test plan, and rollback path. Unrelated cleanup is kept out so cause and effect remain legible.

The conductor does not need to write the code

The scarce work is choosing the question, holding the architecture together, refusing false certainty, and deciding what evidence is strong enough to move forward. Coding is one part of the process, not the conductor's identity.

Required phase ledger

- Objective and owner
- Inputs and assumptions
- Paths and resources claimed
- Acceptance gates
- Artifacts and evidence expected

Fail-closed default

- No silent scope expansion
- No inferred approval
- No claim beyond the observed environment
- No merge on unresolved contradiction
- No promotion without rollback

ONE CYCLE, PHASE TWO

From a candidate to compounding process memory

07 Test at the right layer

Run unit, integration, mutation, adversarial, recovery, and hardware tests that match the claim. Simulation cannot silently stand in for physical qualification.

09 Escalate disagreement

Conflicting conclusions return to the conductor with the evidence on both sides. The dispute is narrowed into a test rather than settled by status, eloquence, or majority vote.

11 Ratify, reject, or reopen

Passing the frozen contract allows a bounded claim. Failure blocks promotion. Later contradictory evidence reopens the milestone without erasing the original record.

08 Invite hostile review

Independent watchers inspect every commit and its comments. They look for hidden assumptions, evidence gaps, stale state, misleading counts, unsafe lifecycles, and claims that outrun results.

10 Issue a receipt

The accepted run binds source identity, binary or artifact identity, machine identity, actual gates, results, and limitations. Evidence is append-only and cannot be rewritten by a later rerun.

12 Feed the next cycle

Successful procedures, failure patterns, review findings, and improved test gates become inputs to later work. The game continues with stronger rules and better evidence.

Learning inside development

The process improves not only the artifact but also the prompts, role boundaries, checklists, tests, review questions, and decomposition strategy used to build the next artifact.

Learning inside AIEN

The intended machine loop follows the same pattern: propose, define, realize, verify, act, record evidence, improve search, and preserve useful procedures.

Succession is the long experiment

Today the human conductor carries the complete objective across tools that each see only part of the work. The research ends only when those coordination functions can be represented, tested, and continued without their original conductor, while independent verification still prevents self-approval.

WHAT TURNS ACTIVITY INTO EVIDENCE

Every claim must carry its own boundary

Gate	Question	Minimum receipt
Identity	Exactly what source, artifact, model, machine, and test definition produced this result?	Content digests, commit identity, machine facts, tool versions.
Semantic	Did the candidate preserve the meaning and constraints of the task?	Typed program identity, invariants, verifier outcome.
Realization	Did the physical implementation actually perform the intended transformation?	Observed execution, outputs, hardware state, error bounds.
Adversarial	What happens under malformed inputs, stale handles, races, replay, interruption, or corruption?	Negative cases, refusal behavior, recovery result.
Durability	Can the result survive restart, rerun, version change, or replacement of a transient tool?	Reconstruction steps, persistence proof, rollback evidence.
Independence	Can another reviewer reproduce or challenge the claim without relying on the author's narrative?	Public specification, exact commands, immutable receipts.

Evidence hierarchy

1. Observed physical execution under a declared machine contract.
2. Reproducible execution in a controlled emulator or simulator.
3. Static audit and formal checks.
4. Code inspection and design argument.
5. Documentation claim.
6. Model output or human expectation.

Promotion rule

Higher evidence does not automatically prove a larger claim. It only strengthens the specific claim tested. A silicon result for one kernel is not general accelerator sovereignty. A verified procedure is not general intelligence.

Correction rule

When a later review finds a problem, preserve the old artifact and receipt, publish the erratum, reopen the gate, and require a new immutable receipt.

The correct response to weak evidence is not stronger language. It is a stronger experiment.

FORTY-ONE BOUNDED MILESTONES

The roadmap moves from awakening to succession

Each era depends on evidence from the one beneath it. Later claims are blocked when a lower gate is reopened.

M0-M3	Foundational	QUALIFIED SCOPE
	Ratify the architecture, awaken from reset, establish machine authority, and account for bounded effects.	
M4-M7	Omega core substrate	COMPLETE
	Represent meaning, generate native instructions, reproduce the minimal compiler, and enforce verification.	
M8-M14	Program synthesis and library learning	COMPLETE
	Search over typed programs, preserve successful procedures, discover abstractions, and realize semantics for a machine graph.	
M15-M19	Accelerator cognition substrate	M19 ACTIVE
	Cross the hardware boundary, characterize native submission, run verified vector and matrix work, and establish persistent accelerator execution.	
M20-M26	Sovereign training runtime	PLANNED
	Build tensors, automatic differentiation, optimizers, Omega-generated training traces, AIEN_0, guided synthesis, and learned abstraction proposals.	
M27-M35	Physics Zero discovery	PLANNED
	Use sealed hidden-law worlds to test theory formation, active experimentation, concept invention, extrapolation, and bounded real-lab work.	
M36-M40	General AIEN	PLANNED
	Add the human interface, resident cognition, continual library learning, scientific autonomy, and governed succession.	

The roadmap's integrity rule

Complete, environment-qualified, reopened, and planned are different claims. M19 shows why better evidence must be able to reverse an earlier conclusion.^[4]

ERA 1 AND ERA 2

Establish the root, then make meaning executable

M0	DOCTRINE_V1	COMPLETE
	Ratify the architectural corpus and the permanent separation of responsibilities. DOCTRINE_V1 is the historical milestone identifier for doctrine version 1; it is unrelated to the “AI Generation V1” terminology used in the lineage section.	
M1	ATLAS_BOOT	QEMU
	Create the irreducible <code>atlas.bin</code> bootstrap and verify its two seams: static reconciliation and fail-closed execution behavior.	
M2	PHYSICS_BOOT	QEMU
	Establish the physical authority nucleus, exception-vector ownership, frame bounds, and root capability.	
M3	PHYSICS_EFFECTS	QEMU
	Create an attenuating capability ledger, bounded effect broker, replay resistance, and sealed execution receipts. M2-M3 preserve their historical milestone names and original scope descriptions. Current responsibility boundaries are defined by the Forge/AEGIS/AIENOS architecture on page 6.	
M4	OMEGA_SEMANTICS	COMPLETE
	Represent typed meaning and invariants with content-addressed identity that does not depend on a substrate.	
M5	OMEGA_AARCH64	COMPLETE
	Generate native AArch64 machine bytes directly, without LLVM, Clang, GCC, or GNU assembler in the realization path.	
M6	OMEGA_SELF_HOST	COMPLETE
	Use Omega's own semantic graph to reproduce the minimal realization compiler.	
M7	OMEGA_VERIFY	COMPLETE
	Require verification stages for structural validity, semantics, and realization, with a framework for proof-carrying extensions.	

What this era proves

A minimal boot path and a formal semantic engine exist at their declared qualification levels. It does not yet prove native Atlas qualification, general-purpose synthesis, sovereign training, or a resident intelligence.

ERA 3

Turn verified programs into a growing procedural library

M8	OMEGA_PROGRAM_CORE	COMPLETE
	Define explicit programs, a synthesis-task contract, and cost models so search has an inspectable object and objective.	
M9	OMEGA_SYNTHESIS_V0	COMPLETE
	Search deterministically over typed base primitives to construct programs that satisfy a bounded task.	
M10	OMEGA_LIBRARY_V1	COMPLETE
	Store verified procedures in a versioned catalog with identity and provenance.	
M11	OMEGA_LIBRARY_DISCOVERY	COMPLETE
	Discover reusable abstractions from the existing program corpus and admit them only after verified reuse.	
M12	OMEGA_LIVING_MATVEC	COMPLETE
	Select among valid realizations as inputs and cache conditions change, using measurement rather than a single frozen implementation.	
M13	OMEGA_MACHINE_GRAPH	COMPLETE
	Represent execution pipelines and memory hierarchies as a formal machine graph supplied by the realization layer.	
M14	OMEGA_REALIZATION_SYNTHESIS	COMPLETE
	Combine semantic and machine graphs to synthesize a realization for declared hardware capabilities.	

The learning object

The core asset is a library of procedures whose meaning, origin, verification, machine context, and measured cost can be inspected.

The next constraint

Software-level synthesis must cross into real accelerator execution without surrendering semantic identity or evidence quality.

ERA 4

Cross the accelerator boundary without relaxing the proof

M15	PHYSICS_ACCELERATOR_LINK	BOUNDARY
	Bound accelerator authority in observed DGX Spark topology: coherent memory, isolation, address translation, and device boundaries.	
M16	BLACKWELL_NATIVE_PATH_KNOWN	REQUALIFIED
	Empirically characterize native Blackwell GB10 submission: memory-mapped control, queues, doorbells, and completions. Requalified without libcuda.	
M17	OMEGA_BLACKWELL_VECTOR	SILICON
	Bind a vector computation's semantic contract to a verified native sm_121 realization and physical DGX Spark execution.	
M18	OMEGA_BLACKWELL_MATMUL	SILICON
	Generate and execute a native Blackwell matrix multiplication realization using tensor acceleration on physical GB10 silicon.	
M19	OMEGA_ACCELERATOR_RESIDENT	REOPENED
	Keep an Omega execution substrate in accelerator-accessible coherent memory with a correct lifecycle, exactly-once completion, bounded resources, and immutable qualification evidence.	

Why M19 was reopened

An independent review found evidence-integrity and runtime-correctness problems after the implementation and a 175-gate run had landed. The corrective program requires receipts that cannot be overwritten, hardware identity that is probed rather than asserted, full allocate-use-revoke-free behavior, race tests, and a long silicon soak with flat resource use.

Success condition

A new immutable receipt demonstrates no known lifecycle or exactly-once defect under the expanded adversarial contract.

Research meaning

The system is not permitted to build M20 on a lower layer whose qualification is known to be misleading.

ERA 5

Build the training machinery, then make guidance earn its place

M20	OMEGA_TENSOR	PLANNED
	Add immutable tensor meaning, shapes, strides, layouts, views, reductions, broadcasting, transpose, and general matrix multiplication.	
M21	OMEGA_AUTODIFF	PLANNED
	Transform an Omega forward graph into an Omega backward graph so gradients follow the same verification and realization path.	
M22	OMEGA_OPTIMIZER	PLANNED
	Implement verified optimizer steps with shadow state, validation, atomic generation changes, and crash recovery that yields old or new, never half-updated.	
M23	OMEGA_SEARCH_GUIDE_TRAINING	PLANNED
	Capture parent-child candidates, transformations, semantic features, verifier outcomes, machine results, cost, and reasons for pruning or success.	
M24	AIEN_0	PLANNED
	Train the first sovereign neural guide to score candidate next actions from the current state, remaining objective, and operation semantics.	
M25	AIEN_GUIDED_SYNTHESIS	PLANNED
	Demonstrate lower search cost or faster time to a valid realization while preserving the same verification acceptance standard.	
M26	AIEN_ABSTRACTION_DISCOVERY	PLANNED
	Let AIEN propose reusable abstractions while Omega admits them only when they preserve semantics and improve held-out reuse or search.	

Sealed evaluation before training

Commit the holdout seed, generator, metrics, thresholds, allowed data, architecture, optimizer, and training budget first. Train once, freeze the model identity, reveal the holdout, evaluate, and write an immutable receipt.

ERA 6, PART ONE

Test discovery where memorized human answers cannot help

Physics Zero is separate from Forge. Forge realizes computation on hardware. Physics Zero asks whether AIEN can discover laws through controlled evidence.

M27	PHYSICS_ZERO_PROTOCOL	PLANNED
Define a contamination firewall, sealed tasks, allowed observations, frozen metrics, and auditable benchmark contracts.		
M28	PHYSICS_ZERO_HIDDEN_WORLDS	PLANNED
Create discrete and continuous worlds with hidden rules and randomized channels that prevent success by naming familiar textbook concepts.		
M29	AIEN_THEORY_DISCOVERY	PLANNED
Synthesize executable theories, compare competing candidates, and evaluate predictive calibration and compression.		
M30	AIEN_ACTIVE_EXPERIMENTATION	PLANNED
Choose experiments whose outcomes distinguish between competing theories instead of collecting observations indiscriminately.		
M31	AIEN_CONCEPT_FORMATION	PLANNED
Invent reusable latent concepts that improve prediction or explanation without requiring human vocabulary.		

Reward

Accurate prediction, calibrated uncertainty, compact explanation, and experiments that separate competing theories.

Do not reward

Similarity to familiar terminology, confident prose, hindsight fit, or theories that cannot risk being wrong.

The contamination test

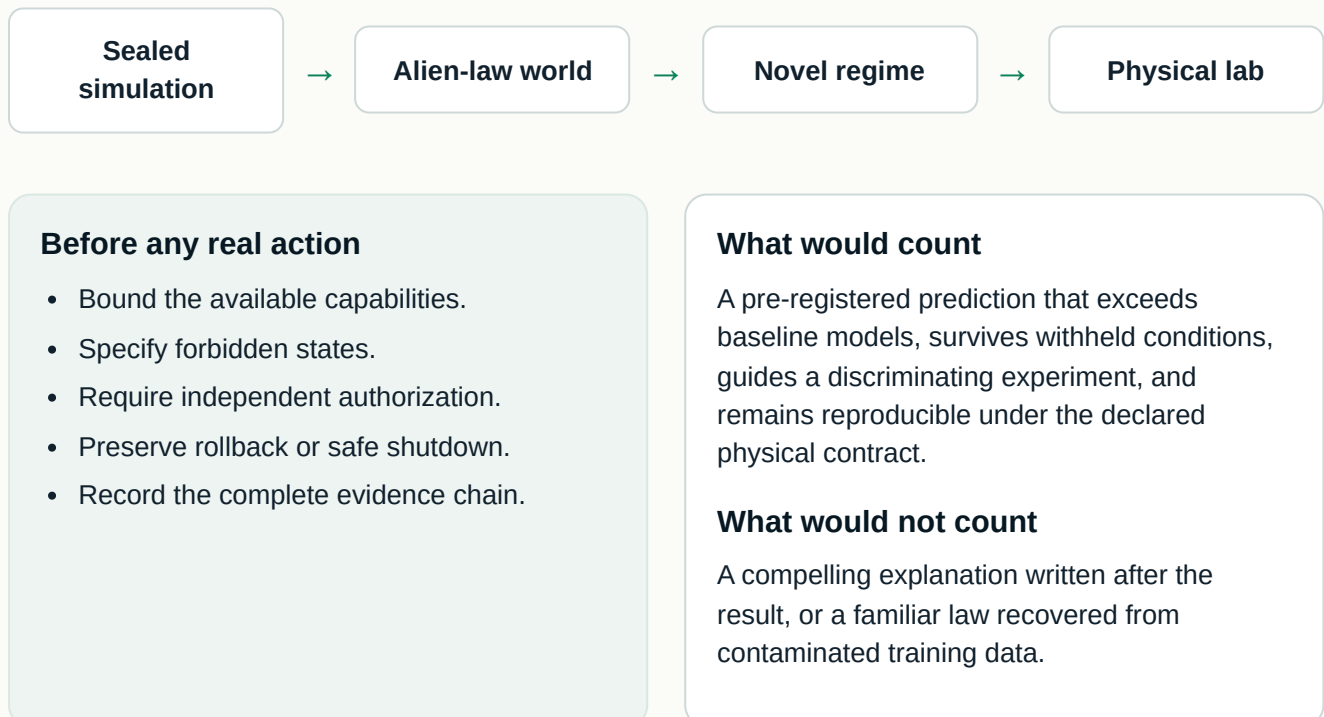
A discovery benchmark is meaningful only when its rules are hidden before evaluation and success cannot come from reproducing human-written solutions seen during training.

ERA 6, PART TWO

Move from hidden worlds toward bounded physical science

M32	PHYSICS_ZERO_ALIEN_WORLDS	PLANNED
	Test scientific discovery in synthetic universes governed by unfamiliar laws, where human physics language is actively unhelpful.	
M33	PHYSICS_ZERO_NOVEL_REGIME	PLANNED
	Require theories to predict behavior in unobserved state regimes rather than interpolate only inside prior observations.	
M34	PHYSICS_ZERO_NOVEL_PHENOMENON	PLANNED
	Predict an unobserved phenomenon, specify the conditions that should produce it, and test whether the phenomenon appears.	
M35	PHYSICS_ZERO_REAL_LAB	PLANNED
	Enter a capability-bounded physical laboratory with explicit safety limits, reversible actions where possible, and independent authorization.	

Escalating burden of proof



Physics Zero is a future research program. No claim of autonomous scientific discovery is made here.

ERA 7

General AIEN culminates in governed succession

M36	AIEN_HUMAN_INTERFACE	PLANNED
Map natural language and perception into the existing semantic and planning architecture. Conversation is an interface, not the architecture itself.		
M37	AIEN_RESIDENT	PLANNED
Sustain a long-lived cognitive loop in accelerator memory with bounded resources and deterministic reconstruction.		
M38	OMEGA_CONTINUAL_LIBRARY_LEARNING	PLANNED
Run a continual Wake, Solve, Verify, Sleep loop that admits useful procedures, measures them, and retires abstractions that no longer earn their cost.		
M39	AIEN_SCIENTIFIC_AUTONOMY	PLANNED
Conduct bounded campaigns of observation, hypothesis, prediction, experiment, falsification, and revision.		
M40	AIEN_SUCCESSION	PLANNED
Allow AIEN-N to design or train an AIEN-N+1 candidate while preventing either from approving the promotion.		

Succession contract

Freeze evaluation first. Run the candidate in isolated worlds and canaries. Require independent verification and operator or governance approval. Preserve a real rollback path. A successor that can waive these conditions has failed the research objective.

The Infinite Game succeeds when the process can continue without its original conductor, not when the conductor simply leaves.

The human role is not removed by hiding it inside a model. It is decomposed into explicit functions: goal definition, context integration, conflict escalation, admission standards, evidence judgment, promotion, and rollback. Each function must become testable before succession is more than a metaphor.

THE NEXT WORK IS ALREADY ORDERED

Repair the foundation before extending the claim



Lane 1: M19R and Forge

- Make qualification receipts immutable and content-addressed.
- Probe machine identity instead of asserting it.
- Correct allocation and revocation lifecycle.
- Prove exactly-once dispatch completion under races.
- Run a long physical-silicon resource soak.
- Establish a typed Omega-to-Forge request boundary.

Lane 2: AIENOS trust

- Finish owner-controlled key hierarchy and sealed storage.
- Enforce anti-rollback and production identity separation.
- Demonstrate deterministic recovery.
- Build minimum native networking.
- Place inference behind the native platform boundary.
- Demonstrate power-cycle continuity for one agent identity.

Lane 3: Omega training substrate

- Close general floating-point semantics.
- Verify CPU and GB10 numerical parity.
- Implement immutable tensors and general operations.
- Generate backward graphs as ordinary Omega.
- Add transactional optimizers.
- Bind training steps to reproducible provenance.

Lane 4: Composition

- Define one capability graph for tools, models, machines, and constraints.
- Separate reusable skills from atomic tools.
- Branch candidate worlds before external action.
- Commit only the selected verified world.
- Add J-Space search and Fabric placement interfaces.
- Keep recursive improvement off the critical path until boundaries stabilize.

Convergence gate

These lanes may progress in parallel, but they converge before AIEN_0 enters the live trusted path. A learned guide must not sit on top of known-misleading evidence or unstable execution boundaries.

HOW THE HYPOTHESIS CAN LOSE

Success requires measurable advantage without weaker truth

Lineage size	Count permanent runtime dependencies, source lines in the trusted bare-metal lineage, binary size, and unreconstructable vendor components. The target is a shrinking, inspectable base, not a cosmetic rewrite.
Provenance	Measure the share of accepted artifacts whose source, binary, machine, test contract, results, and reviewer trail are fully bound into immutable receipts.
Correctness	Track semantic-equivalence pass rates, verifier false acceptance, verifier false rejection, numerical error, and adversarial refusal behavior.
Search value	Compare unguided Omega search with AIEN_0 on frozen held-out tasks: nodes explored, wall-clock time, energy, valid-program rate, and quality of the best realization.
Library value	Measure reuse across held-out task families, compression of solution descriptions, speedup from learned abstractions, and retirement rate for abstractions that stop helping.
Machine value	Compare valid realizations on latency, throughput, memory, energy, thermal stability, portability, and recovery after failure.
Review value	Count independently discovered defects, severity, time to correction, recurrence, and correlated misses across AI providers.
Succession value	Test whether a candidate successor improves frozen objectives while preserving safety, provenance, rollback, and independent promotion controls.

Falsification examples

The central hypothesis weakens if program libraries do not generalize, if AIEN_0 cannot improve search under equal verification, if the trusted lineage cannot be reconstructed without the old stack, or if independent review repeatedly fails to detect consequential defects.

WHAT THIS WORK DOES NOT YET PROVE

The hard part is not building a loop. It is keeping the loop honest.

Risk	Why it matters	Research response
Correlated AI error	Different models can repeat the same assumption because they share data, conventions, or infrastructure.	Use diverse reviewers, adversarial prompts, reproducible tests, and machine evidence. Never treat consensus as proof.
Receipt theater	A receipt can look rigorous while binding the wrong source, wrong machine, or a misleading test count.	Make receipts append-only, derive counts from observed runs, probe hardware identity, and separate qualification from candidate code.
Simulation overreach	QEMU can verify logic while missing real timing, device, firmware, and silicon behavior.	Label scope explicitly and maintain separate native gates.
Search explosion	Program synthesis can become computationally impractical as tasks grow.	Use typed constraints, cost models, reusable abstractions, machine feedback, and a learned guide that must prove net value.
Verifier bugs	The checker is software and can be wrong.	Minimize it, test it adversarially, separate layers, use independent implementations where valuable, and audit the evidence path.
Hidden foreign dependency	A construction tool can accidentally become a permanent runtime ancestor.	Maintain a lineage inventory and prove clean-machine reconstruction without the foreign component.
Self-improvement bypass	A system that can modify or promote itself can silently weaken its tests.	Freeze evaluation first, isolate candidates, require independent promotion, and preserve rollback.

Current limitation

M19 is reopened. The sovereign training path is not built. AIEN_0 has not been trained. General discovery and succession remain research targets.

Communication limit

No worldwide first is claimed. Novelty should be argued through exact architecture, code, tests, and results, not a superlative that cannot be audited.

The existence of risks is not a reason to abandon the program. It is a reason to write them into the acceptance gates before success is declared.

AIEN V2 STARTS AT ATLAS

Construction does not imply dependency

Today's AI systems are part of the workshop. They help interpret objectives, write code, review changes, propose tests, and expose contradictions. Their participation is recorded in provenance. That does not automatically make their models, services, operating systems, or runtimes permanent ancestors of the machine being built.

AI Generation V1

Contemporary AI construction tools used by The Infinite Game.

AIEN Classic

Pre-Atlas implementations, experiments, and the older Rust AI stack.

AIEN V2

The sovereign lineage that begins at Atlas and must eventually reconstruct its trusted path.

V1 and V2 are AIEN project-generation terms. They do not claim to redefine the universal history or generations of artificial intelligence.

M0 doctrine predates the executable genesis; Atlas M1 is the beginning of V2's trusted runtime ancestry.

V1 tools build**source + evidence****AIEN V2 lineage****Allowed**

- Use contemporary AI as a construction instrument.
- Use incumbent compilers and operating systems as temporary oracles.
- Record all such use honestly in provenance.
- Compare the sovereign result against frozen references.

Not allowed at closure

- Require a hosted model to boot, verify, recover, or rebuild the trusted path.
- Hide a vendor runtime beneath a renamed interface.
- Erase the history of how the system was constructed.
- Call the lineage sovereign while a foreign dependency remains permanent.

Why the distinction matters

A telescope can help build a map without becoming part of the territory. In the same way, a V1 model can help build AIEN V2 while remaining outside the installed lineage, provided the resulting system can be inspected, reproduced, and operated without that model.

Repository moves and naming changes remain subject to the canonical architecture and current execution plan.

HOW THIS WORK SHOULD BE COMMUNICATED

Publish the boundary with the result

Say what was observed

Name the artifact, environment, test contract, and result. Link the code or receipt. Explain what a non-programmer should understand from it.

Say what was not observed

If hardware qualification is pending, say so. If a model is planned, do not write as though it exists. If evidence was reopened, use the current status.

Show the correction path

Publish errata and the gate that will close them. A transparent reversal is more credible than an uninterrupted success story.

Invite falsification

State what result would disprove the claim and provide enough information for another person to reproduce the experiment.

Current public position

AIEN has a real, inspectable foundation and a coherent research route. It has not yet produced sovereign training, AIEN_0, general intelligence, autonomous scientific discovery, or succession. The accomplishment is not that the final claim has been reached. It is that the path is being converted into explicit layers, testable milestones, working artifacts, independent reviews, and evidence that can say no.

Verify everything. Trust no single entity. Learn from evidence.

The deepest continuity is between the method and the machine. The Infinite Game uses many minds, separates proposal from judgment, records disagreement, and compounds what survives review. AIEN is being designed to do the same. The conductor's present task is to make each hidden act of coordination explicit enough that it can eventually be verified, inherited, and continued.

That is why the process has no finish line. Build. Verify. Learn. Build better. Then succession, with the same safeguards still in force.

PUBLIC AUTHORITIES USED FOR THIS EDITION

Sources and repository snapshot

- [1] **Plan Authority.** Defines the three whole-system authorities, current Forge terminology, and the rule that implementation evidence outranks planning claims.
https://github.com/aien-dev/aien-architecture/blob/main/PLAN_AUTHORITY.md
- [2] **Current Execution Plan.** Defines the current critical path, corrective M19 work, Forge boundary, AIENOS trust work, Omega training substrate, composition work, sovereign closure, and final release gate.
https://github.com/aien-dev/aien-architecture/blob/main/CURRENT_EXECUTION_PLAN.md
- [3] **Canonical Architecture.** Defines Atlas, AIEN, Omega, Forge, Aegis, hardware, evidence, and the separation between semantic meaning and physical realization.
<https://github.com/aien-dev/aien-architecture/blob/main/doctrine/ARCHITECTURE.md>
- [4] **Canonical Roadmap.** Defines M0 through M40, official status vocabulary, evidence links, qualification scopes, the M19 reopening, and all planned milestones.
<https://github.com/aien-dev/aien-architecture/blob/main/doctrine/ROADMAP.md>
- [5] **Public repository organization.** Entry point to the broader open repositories and project history.
<https://github.com/aien-dev>

Edition snapshot

Repository heads observed in fresh public clones on 2026-09-27:

Repository	Observed commit	Role in this brief
aien-architecture	1f98d0a60c09	Authority, doctrine, roadmap, execution plan
omega	ae2476d2c16b	Semantic, synthesis, verification, accelerator evidence
physics	a477aeef7005	Historical and current machine-realization implementation location
atlas	0ec17d4b45c9	Minimal bootstrap artifact and qualification evidence
aienos	07bdb86de0ac	Trusted operating substrate work

Reading rule

Milestone status changes over time. For current status, read the canonical roadmap first. For what should be built next, read the current execution plan. For what is actually implemented, inspect code and evidence at the cited commit.

Read published posts and download this brief at <https://www.aienos.com>.